CSCI 432 Handout 15: Union Find Algorithm

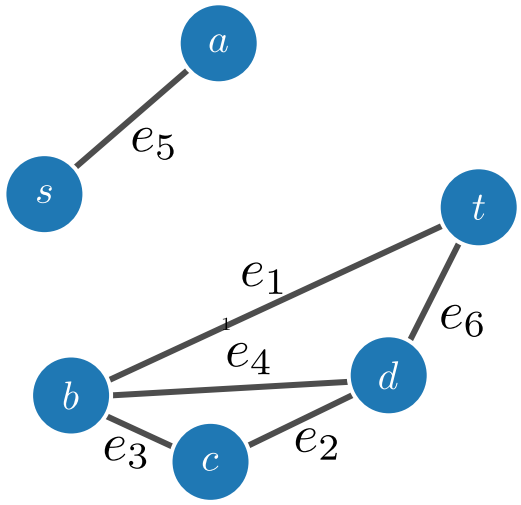
Name: _____

7 April 2026

Graphs

1. If we have a VERTEX object, what sorts of data/attributes might it have?.
   Answer

2. If we have an EDGE object, what sorts of data/attributes might it have?.
   Answer



3. Show how to store the graph above as an incidence matrix ...

   Answer

4. ... and as an adjacency list.

   Answer

Counting Connected Components

Algorithm 1 Count Components

Input: A graph $G = (V, E)$
Output: The number of connected components in G.

```
Initialize S, a union-find data structure with the vertex set V
E' ← E
while E' ≠ ∅ do
    e ← E'.pop()
    s_1 ← S.find(e.head)
    s_2 ← S.find(e.tail)
    if s_1 ≠ s_2 then
        S.union(s_1, s_2)
    end if
end while
return S.size()
```

1. In Algorithm 1, S is a data structure that keeps track of the connected components of a graph that is initially just the vertex set, and builds up to the full graph G by adding one vertex at a time. Without getting into more specifics, walk through running this algorithm for the example graph on the previous page. (We'll look into implementation details next, for now, just think 'what are the connected components' at each step as you are walking through).

   Answer

2. Quick Find: Suppose S keeps track of connected components in an array A. Here, $A[i]$ would store the id of the component for vertex $V[i]$. What would the time complexity of the union and find operations be with this implementation? Walk through using this data structure on the example graph.

   Answer

3. Quick union: Suppose S stores the connected components as rooted trees. Here, each vertex would be a node in a (possibly one-vertex) tree, and the id of the connected component is the label of the root of that tree. What would the time complexity of the union and find operations be with this implementation? Walk through using this data structure on the example graph.

   Answer

Two Heuristics

Using the Fast union approach, we add two heuristics for improved runtime!

1. Depth: Associate a variable named 'rank' for each rooted tree. When unioning (merging) two rooted trees together, add the smaller to the larger (as measured by the rank variable). If there is a tie, pick arbitrarily and add one to the rank. Give this a try with the example graph.

   Answer

2. Fatten: When running a find operation (going up the parent until I find the root), directly connect each vertex encountered to the root. Give this a try with the example graph, using both the rank and fatten improvements.

   Answer